Frequentist analysis using the bag-of-words approach: forces and limitations

Lino Galiana
Version of October 20, 2026

This chapter takes a closer look at frequentist analysis applied to textual data by presenting the challenges in the field of information retrieval. After presenting the cleaning and vectorisation of texts using TF-IDF, it discusses the limitations of the *bag of words* approach and paves the way for *embeddings*, vectors representing textual objects by proximity of meaning, the subject of the next chapter.

⚠️ **Automatically generated PDF.** This document is produced from the course website and may contain formatting issues. For an up-to-date version adapted to your reading, visit the course page: pythonds.linogaliana.fr.

Table of contents

1 Introduction **2**
1.1 Data 2
2 The TF-IDF Measure (*term frequency - inverse document frequency*) **2**
2.1 The Document-Term Matrix 2
2.2 Use for Information Retrieval 3
2.3 Example 4
2.4 Application 5
3 An Initial Enhancement of the Bag-of-Words Approach: *n-grams* **12**
4 Some Applications **14**
4.1 Named Entity Recognition 14
4.2 Text Data Classification: The `Fasttext` Algorithm 15
Informations additionnelles 16
Bibliography **18**

To move forward in this chapter, we need to perform some preliminary installations:

```
!pip install spacy
!python -m spacy download en_core_web_sm
!python -m spacy download fr_core_news_sm
```

It is also useful to define the following function, taken from our previous chapter:

```
def clean_text(doc):
    # Tokenize, remove stop words and punctuation, and lemmatize
    cleaned_tokens = [token.lemma_ for token in doc if not token.is_stop and not token.is_punct]
    # Join tokens back into a single string
    cleaned_text = ' '.join(cleaned_tokens)
    return cleaned_text
```

1 Introduction

Previously, we saw the importance of cleaning data to filter down the volume of information present in unstructured data. The goal of this chapter is to deepen our understanding of the frequency-based approach applied to text data. We will explore how this frequentist analysis helps summarize the information contained within a text corpus. We'll also look at how to refine the *bag of words* approach by taking into account the order or proximity of terms within a sentence.

1.1 Data

We will reuse the Anglo-Saxon dataset from the previous chapter, which includes texts from gothic authors Edgar Allan Poe (*EAP*), HP Lovecraft (*HPL*), and Mary Wollstonecraft Shelley (*MWS*).

```
import pandas as pd

url='https://github.com/GU4243-ADS/spring2018-project1-ginnyqg/raw/master/data/spooky.csv'
#1. Import des données
horror = pd.read_csv(url,encoding='latin-1')
#2. Majuscules aux noms des colonnes
horror.columns = horror.columns.str.capitalize()
#3. Retirer le prefixe id
horror['ID'] = horror['Id'].str.replace("id","")
horror = horror.set_index('Id')
```

2 The TF-IDF Measure (*term frequency - inverse document frequency*)

2.1 The Document-Term Matrix

As mentioned earlier, we construct a synthetic representation of our corpus as a bag of words, where words are sampled more or less frequently depending on their appearance frequency. This is, of course, a simplified representation of reality: word sequences are not just random independent words.

However, before addressing those limitations, we should complete the bag-of-words approach. The most characteristic representation of this paradigm is the document-term matrix, mainly used to compare corpora. It involves creating a matrix where each document is represented by the presence or absence of terms in our corpus. The idea is to count how often words (terms, in columns) appear in each sentence or phrase (documents, in rows). This matrix then becomes a numerical representation of the text data.

Consider a corpus made up of the following three sentences:

- The practice of knitting and crocheting
- Passing on the passion for stamps
- Living off one's passion"

The corresponding document-term matrix is:

Sentence	and	crocheting	for	knitting	living	of	one's	on	passion	passing	practice	stamps	the
The practice of knitting and crocheting	1	1	0	1	0	1	0	0	0	0	1	0	1
Passing on the passion for stamps	0	0	1	0	0	0	0	1	1	1	0	1	1
Living off one's passion	0	0	0	0	1	0	1	0	1	0	0	0	0

Each sentence in the corpus is associated with a numeric vector. For instance, the sentence *"La pratique du tricot et du crochet"*, which is meaningless to a machine on its own, becomes a numeric vector it can interpret: `[1, 0, 2, 1, 1, 0, 1, 0, 0, 0, 1, 0]`. This numeric vector is a *sparse* representation of language, since each document (row) will only contain a small portion of the total vocabulary (all columns). Words that do not appear in a document are represented as zeros, hence a *sparse* vector. As we'll see later, this numeric representation is very different from modern *embedding* approaches, which are based on dense representations.

2.2 Use for Information Retrieval

Different documents can then be compared based on these measures. This is one of the methods used by search engines, although the most advanced ones rely on far more sophisticated approaches. The tf-idf metric (*term frequency–inverse document frequency*) allows for calculating a relevance score between a search term and a document using two components:

$$\text{tf-idf}(t, d, D) = \text{tf}(t, d) \times \text{idf}(t, D)$$

Let t be a specific term (e.g., a word), d a specific document, and D the entire set of documents in the corpus.

- The `tf` component computes a function that increases with the frequency of the search term in the document under consideration;
- The `idf` component computes a function that decreases with the frequency of the term across the entire document set (or corpus).
- The first part (*term frequency*, TF) is the frequency of occurrence of term t in document d. There are normalization strategies available to avoid biasing the score in favor of longer documents.

$$\text{tf}(t, d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}}$$

where $f_{t,d}$ is the raw count of how many times term t appears in document d, and the denominator is the total number of terms in document d.

- The second part (*inverse document frequency*, IDF) measures the rarity—or conversely, the commonness—of a term across the corpus. If N is the total number of documents in the corpus D, this part of the metric is given by

$$\text{idf}(t, D) = \log\left(\frac{N}{|\{d \in D : t \in d\}|}\right)$$

The denominator ($|\{d \in D : t \in d\}|$) corresponds to the number of documents in which the term t appears. The rarer the word, the more its presence in a document is given additional weight.

Many search engines use this logic to find the most relevant documents in response to a search query. One notable example is `ElasticSearch`, the software used to implement powerful search engines. To rank the most relevant documents for a given search term, it uses the BM25 distance metric, which is a more advanced version of the TF-IDF measure.

2.3 Example

Let's illustrate this with a small corpus. The following code implements a TF-IDF metric. It slightly deviates from the standard definition to avoid division by zero.

```
import numpy as np

# Documents d'exemple
documents = [
    "Le corbeau et le renard",
    "Rusé comme un renard",
    "Le chat est orange comme un renard"
]

# Tokenisation
def preprocess(doc):
    return doc.lower().split()

tokenized_docs = [preprocess(doc) for doc in documents]

# Term frequency (TF)
def term_frequency(term, tokenized_doc):
    term_count = tokenized_doc.count(term)
    return term_count / len(tokenized_doc)

# Inverse document frequency (DF)
def document_frequency(term, tokenized_docs):
    return sum(1 for doc in tokenized_docs if term in doc)

# Calculate inverse document frequency (IDF)
def inverse_document_frequency(word, corpus):
    # Normalisation avec + 1 pour éviter la division par zéro
    count_of_documents = len(corpus) + 1
    count_of_documents_with_word = sum([1 for doc in corpus if word in doc]) + 1
    idf = np.log10(count_of_documents/count_of_documents_with_word) + 1
    return idf

# Calculate TF-IDF scores in each document
def tf_idf_term(term):
  tf_idf_scores = pd.DataFrame(
    [
      [
      term_frequency(term, doc),
      inverse_document_frequency(term, tokenized_docs)
      ] for doc in tokenized_docs
    ],
    columns = ["TF", "IDF"]
  )
  tf_idf_scores["TF-IDF"] = tf_idf_scores["TF"] * tf_idf_scores["IDF"]
  return tf_idf_scores
```

Let's begin by computing the TF-IDF score of the word "cat" for each document. Naturally, it is the third document—the only one where the word appears—that has the highest score:

```
tf_idf_term("chat")
```

	TF	IDF	TF-IDF
0	0.000000	1.30103	0.000000
1	0.000000	1.30103	0.000000
2	0.142857	1.30103	0.185861

What about the term “renard” (fox in French) which appears in all the documents (making the idf component equal to 1)? In this case, the document where the word appears most frequently—in this example, the second document—has the highest score.

```
tf_idf_term("renard")
```

	TF	IDF	TF-IDF
0	0.200000	1.0	0.200000
1	0.250000	1.0	0.250000
2	0.142857	1.0	0.142857

2.4 Application

The previous example didn’t scale very well. Fortunately, `Scikit` provides an implementation of TF-IDF vector search, which we can explore in a new exercise.

Exercise 1: TF-IDF Frequency Calculation

1. Use the TF-IDF vectorizer from `scikit-learn` to transform your corpus into a `document x terms` matrix. Use the `stop_words` option to avoid inflating the matrix size. Name the model `tfidf` and the resulting dataset `tfs`.
2. After constructing the document x terms matrix with the code below, find the rows where terms matching `abandon` are non-zero.
3. Identify the 50 excerpts where the TF-IDF score for the word *“fear”* is highest and their associated authors. Determine the distribution of authors among these 50 documents.
4. Inspect the top 10 scores where TF-IDF for *“fear”* is highest.

Hint for question 2

```
feature_names = tfidf.get_feature_names_out()
corpus_index = [n for n in list(tfidf.vocabulary_.keys())]
horror_dense = pd.DataFrame(tfs.todense(), columns=feature_names)
```

The vectorizer obtained at the end of question 1 is as follows:

	stop_words **stop_words: {'english'}, list, default=None** **If a string, it is passed to _check_stop_list and the appropriate stop** **list is returned. 'english' is currently the only supported string** **value.** **There are several known issues with 'english' and you should**	['can', 'as', ...]

	consider an alternative (see :ref:`stop_words`). **If a list, that list is assumed to contain stop words, all of which** **will be removed from the resulting tokens.** **Only applies if ``analyzer == 'word'``.** **If None, no stop words will be used. In this case, setting `max_df`** **to a higher value, such as in the range (0.7, 1.0), can automatically detect** **and filter stop words based on intra corpus document frequency of terms.**	
	input input: {'filename', 'file', 'content'}, default='content' - If `'filename'`, the sequence passed as an argument to fit is expected to be a list of filenames that need reading to fetch the raw content to analyze. - If `'file'`, the sequence items must have a 'read' method (file-like object) that is called to fetch the bytes in memory. - If `'content'`, the input is expected to be a sequence of items that can be of type string or byte.	'content'
	encoding encoding: str, default='utf-8' If bytes or files are given to analyze, this encoding is used to decode.	'utf-8'
	decode_error decode_error: {'strict', 'ignore', 'replace'}, default='strict' Instruction on what to do if a byte sequence is given to analyze that contains characters not of the given `encoding`. By default, it is	'strict'

	'strict', meaning that a UnicodeDecodeError will be raised. Other values are 'ignore' and 'replace'.	
	strip_accents strip_accents: {'ascii', 'unicode'} or callable, default=None Remove accents and perform other character normalization during the preprocessing step. 'ascii' is a fast method that only works on characters that have a direct ASCII mapping. 'unicode' is a slightly slower method that works on any characters. None (default) means no character normalization is performed. Both 'ascii' and 'unicode' use NFKD normalization from :func:`unicodedata.normalize`.	None
	lowercase lowercase: bool, default=True Convert all characters to lowercase before tokenizing.	True
	preprocessor preprocessor: callable, default=None Override the preprocessing (string transformation) stage while preserving the tokenizing and n-grams generation steps. Only applies if ``analyzer`` is not callable.	None
	tokenizer tokenizer: callable, default=None Override the string tokenization step while preserving the preprocessing and n-grams generation steps. Only applies if ``analyzer == 'word'``.	None
	analyzer analyzer: {'word', 'char', 'char_wb'} or callable, default='word' Whether the feature should be made of word or character n-grams. Option 'char_wb' creates char-	'word'

	acter n-grams only from text inside word boundaries; n-grams at the edges of words are padded with space. If a callable is passed it is used to extract the sequence of features out of the raw, unprocessed input. .. versionchanged:: 0.21 Since v0.21, if ``input`` is ``'filename'`` or ``'file'``, the data is first read from the file and then passed to the given callable analyzer.	
	token_pattern token_pattern: str, default=r"(?u)\\b\\w\\w+\\b" Regular expression denoting what constitutes a "token", only used if ``analyzer == 'word'``. The default regexp selects tokens of 2 or more alphanumeric characters (punctuation is completely ignored and always treated as a token separator). If there is a capturing group in token_pattern then the captured group content, not the entire match, becomes the token. At most one capturing group is permitted.	'(?u)\\b\\w\\w+\\b'
	ngram_range ngram_range: tuple (min_n, max_n), default=(1, 1) The lower and upper boundary of the range of n-values for different n-grams to be extracted. All values of n such that min_n <= n <= max_n will be used. For example an ``ngram_range`` of ``(1, 1)`` means only unigrams, ``(1, 2)`` means unigrams and bigrams, and ``(2, 2)`` means	(1, ...)

	only bigrams. Only applies if ``analyzer`` is not callable.	
	max_df max_df: float or int, default=1.0 When building the vocabulary ignore terms that have a document frequency strictly higher than the given threshold (corpus-specific stop words). If float in range [0.0, 1.0], the parameter represents a proportion of documents, integer absolute counts. This parameter is ignored if vocabulary is not None.	1.0
	min_df min_df: float or int, default=1 When building the vocabulary ignore terms that have a document frequency strictly lower than the given threshold. This value is also called cut-off in the literature. If float in range of [0.0, 1.0], the parameter represents a proportion of documents, integer absolute counts. This parameter is ignored if vocabulary is not None.	1
	max_features max_features: int, default=None If not None, build a vocabulary that only consider the top `max_features` ordered by term frequency across the corpus. Otherwise, all features are used. This parameter is ignored if vocabulary is not None.	None
	vocabulary vocabulary: Mapping or iterable, default=None Either a Mapping (e.g., a dict) where keys are terms and values are indices in the feature matrix, or	None

	an iterable over terms. If not given, a vocabulary is determined from the input documents.	
	binary binary: bool, default=False If True, all non-zero term counts are set to 1. This does not mean outputs will have only 0/1 values, only that the tf term in tf-idf is binary. (Set `binary` to True, `use_idf` to False and `norm` to None to get 0/1 outputs).	False
	dtype dtype: dtype, default=float64 Type of the matrix returned by fit_transform() or transform().	<class 'numpy.float64'>
	norm norm: {'l1', 'l2'} or None, default='l2' Each output row will have unit norm, either: - 'l2': Sum of squares of vector elements is 1. The cosine similarity between two vectors is their dot product when l2 norm has been applied. - 'l1': Sum of absolute values of vector elements is 1. See :func:`~sklearn.preprocessing.normalize`. - None: No normalization.	'l2'
	use_idf use_idf: bool, default=True Enable inverse-document-frequency reweighting. If False, idf(t) = 1.	True
	smooth_idf smooth_idf: bool, default=True Smooth idf weights by adding one to document frequencies, as if an extra document was seen containing every term in the collection exactly once. Prevents zero divisions.	True

sublinear_tf	sublinear_tf: bool, default=False Apply sublinear tf scaling, i.e. replace tf with 1 + log(tf).

```
/__w/python-datascientist/python-datascientist/.venv/lib/python3.13/site-packages/sklearn/
feature_extraction/text.py:412: UserWarning: Your stop_words may be inconsistent with your
preprocessing. Tokenizing the stop words generated tokens ['ll', 've'] not in stop_words.
  warnings.warn(
```

	aaem	ab	aback	abaft	abandon	abandoned	abandoning	abandonment	abaout	abased	...	zodiacal	zoilus	zokkar	zone	zones	zopyrus	zorry	zubmizzion	zuro	á¼
0	0.0	0.0	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
1	0.0	0.0	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
2	0.0	0.0	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
3	0.0	0.0	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
4	0.0	0.0	0.0	0.0	0.0	0.267616	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

The lines where the term *“abandon”* appears are as follows (question 2):

```
Index([    4,   116,   215,   571,   839,  1042,  1052,  1069,  2247,  2317,
        2505,  3023,  3058,  3245,  3380,  3764,  3886,  4425,  5289,  5576,
        5694,  6812,  7500,  9013,  9021,  9077,  9560, 11229, 11395, 11451,
       11588, 11827, 11989, 11998, 12122, 12158, 12189, 13666, 15259, 16516,
       16524, 16759, 17547, 18019, 18072, 18126, 18204, 18251],
      dtype='int64')
```

The document-term matrix associated with these is as follows:

	aaem	ab	aback	abaft	abandon	abandoned	abandoning	abandonment	abaout	abased	...	zodiacal	zoilus	zokkar	zone	zones	zopyrus	zorry	zubmizzion	zuro	á¼
4	0.0	0.0	0.0	0.0	0.000000	0.267616	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
116	0.0	0.0	0.0	0.0	0.000000	0.359676	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
215	0.0	0.0	0.0	0.0	0.249090	0.000000	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
571	0.0	0.0	0.0	0.0	0.000000	0.153280	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0
839	0.0	0.0	0.0	0.0	0.312172	0.000000	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

Here we notice the drawback of not applying stemming. Variations of *“abandon”* are spread across many columns. *“abandoned”* is treated as different from *“abandon”* just as it is from *“fear”*. This is one of the limitations of the *bag of words* approach.

```
Text    50
dtype: int64
```

The 10 highest scores are as follows:

```
['We could not fear we did not.',
 '"And now I do not fear death.',
 'Be of heart and fear nothing.',
 'Indeed I had no fear on her account.',
 'I smiled, for what had I to fear?',
 'I did not like everything about what I saw, and felt again the fear I had had.',
 'At length, in an abrupt manner she asked, "Where is he?" "O, fear not," she continued, "fear
not that I should entertain hope Yet tell me, have you found him?',
 'I have not the slightest fear for the result.',
 '"I fear you are right there," said the Prefect.']
```

We observe that the highest scores correspond either to short excerpts where the word appears once, or to longer excerpts where the word *“fear”* appears multiple times.

3 An Initial Enhancement of the Bag-of-Words Approach: *n-grams*

We previously identified two main limitations of the bag-of-words approach: its disregard for context and its sparse representation of language, which sometimes leads to weak similarity matches between texts. However, within the bag-of-words paradigm, it is possible to account for the sequence of tokens using *n-grams*.

To recap, in the traditional *bag of words* approach, word order doesn't matter. A text is treated as a collection of words drawn independently, with varying frequencies based on their occurrence probabilities. Drawing a specific word doesn't affect the likelihood of subsequent words.

A way to introduce relationships between sequences of *tokens* is through *n-grams*. This method considers not only word frequencies but also which words follow others. It's particularly useful for disambiguating homonyms. The computation of *n-grams*[1] is the simplest method for incorporating context.

To carry out this type of analysis, we need to download an additional corpus:

```
import nltk
nltk.download('genesis')
nltk.corpus.genesis.words('english-web.txt')
```

```
[nltk_data] Downloading package genesis to /github/home/nltk_data...
[nltk_data]   Package genesis is already up-to-date!
```

```
['In', 'the', 'beginning', 'God', 'created', 'the', ...]
```

`NLTK` provides methods for incorporating context. To do this, we compute n-grams—that is, sequences of n consecutive word co-occurrences. Generally, we limit ourselves to bigrams or at most trigrams:

- Classification models, sentiment analysis, document comparison, etc., that rely on n-grams with large n quickly face sparse data issues, reducing their predictive power;
- Performance drops quickly as n increases, and data storage costs increase substantially (roughly n times larger than the original dataset).

Let's quickly examine the context in which the word `fear` appears in the works of Edgar Allan Poe (EAP). To do this, we first transform the EAP corpus into `NLTK` tokens:

```
eap_clean = horror.loc[horror["Author"] == "EAP"]
eap_clean = ' '.join(eap_clean['Text'])
tokens = eap_clean.split()
print(tokens[:10])
text = nltk.Text(tokens)
print(text)
```

```
['This', 'process,', 'however,', 'afforded', 'me', 'no', 'means', 'of', 'ascertaining', 'the']
<Text: This process, however, afforded me no means of...>
```

You will need the functions `BigramCollocationFinder.from_words` and `BigramAssocMeasures.likelihood_ratio`:

[1] We use the term *bigrams* for two-word co-occurrences, *trigrams* for three-word ones, etc.

Exercise 2: n-grams and the Context of the Word “fear”

1. Use the `concordance` method to display the context in which the word `fear` appears.
2. Select and display the top collocations, for instance using the likelihood ratio criterion.

When two words are strongly associated, it may be due to their rarity. Therefore, it’s often necessary to apply filters—for example, ignore bigrams that occur fewer than 5 times in the corpus.

3. Repeat the previous task using the `BigramCollocationFinder` model, followed by the `apply_freq_filter` method to retain only bigrams appearing at least 5 times. Then, instead of the likelihood ratio, test the method `nltk.collocations.BigramAssocMeasures().jaccard`.
4. Focus only on *collocations* involving the word *fear*.

Using the `concordance` method (question 1), the list should look like this:

```
Exemples d'occurences du terme 'fear' :
Displaying 13 of 13 matches:
d quick unequal spoken apparently in fear as well as in anger. What he said wa
hutters were close fastened, through fear of robbers, and so I knew that he co
to details. I even went so far as to fear that, as I occasioned much trouble,
years of age, was heard to express a fear "that she should never see Marie aga
ich must be entirely remodelled, for fear of serious accident I mean the steel
 my arm, and I attended her home. 'I fear that I shall never see Marie again.'
clusion here is absurd. "I very much fear it is so," replied Monsieur Maillard
bt of ultimately seeing the Pole. "I fear you are right there," said the Prefe
er occurred before.' Indeed I had no fear on her account. For a moment there w
erhaps so," said I; "but, Legrand, I fear you are no artist. It is my firm int
 raps with a hammer. Be of heart and fear nothing. My daughter, Mademoiselle M
e splendor. I have not the slightest fear for the result. The face was so far
arriers of iron that hemmed me in. I fear you have mesmerized" adding immediat
```

Although it is easy to see the words that appear before and after, this list is rather hard to interpret because it combines a lot of information.

`Collocation` involves identifying bigrams that frequently occur together. Among all observed word pairs, the idea is to select the “best” ones based on a statistical model. Using this method (question 2), we get:

```
[('of', 'the'),
 ('in', 'the'),
 ('had', 'been'),
 ('to', 'be'),
 ('have', 'been'),
 ('I', 'had'),
 ('It', 'was'),
 ('it', 'is'),
 ('could', 'not'),
 ('from', 'the'),
 ('upon', 'the'),
 ('more', 'than'),
 ('it', 'was'),
 ('would', 'have'),
 ('with', 'a'),
 ('did', 'not'),
 ('I', 'am'),
 ('the', 'a'),
```

```
 ('at', 'once'),
 ('might', 'have')]
```

If we model the best collocations:

```
"Gad Fly"
'Hum Drum,'
'Rowdy Dow,'
Brevet Brigadier
BarriÃ¨re du
ugh ugh
Ourang Outang
Chess Player
John A.
A. B.
hu hu
General John
'Oppodeldoc,' whoever
mille, mille,
Brigadier General
```

This list is a bit more meaningful, including character names, places, and frequently used expressions (like *Chess Player* for example).

As for the *collocations* of the word *fear*:

```
[('fear', 'of'), ('fear', 'God'), ('I', 'fear'), ('the', 'fear'), ('The', 'fear'), ('fear',
'him'), ('you', 'fear')]
```

If we perform the same analysis for the term *love*, we logically find subjects that are commonly associated with the verb:

```
[('love', 'me'), ('love', 'he'), ('will', 'love'), ('I', 'love'), ('love', ','), ('you', 'love'),
('the', 'love')]
```

4 Some Applications

We just discussed an initial application of the *bag of words* approach: grouping texts based on shared terms. However, this is not the only use case. We will now explore two additional applications that lead us toward language modeling: named entity recognition and classification.

4.1 Named Entity Recognition

Named Entity Recognition (NER) is an information extraction technique used to identify the type of certain terms in a text, such as locations, people, quantities, etc.

To illustrate this, let's return to *The Count of Monte Cristo* and examine a short excerpt from the work to see how named entity recognition operates:

```
import requests
import re
```

```
url = "https://www.gutenberg.org/files/17989/17989-0.txt"
response = requests.get(url)
response.encoding = 'utf-8'  # Ensures proper decoding
raw = response.text

dumas = (
  raw
  .split("*** START OF THE PROJECT GUTENBERG EBOOK 17989 ***")[1]
  .split("*** END OF THE PROJECT GUTENBERG EBOOK 17989 ***")[0]
)


def clean_text(text):
    text = text.lower() # convert words to lowercase
    text = " ".join(text.split())
    return text

dumas = clean_text(dumas)

dumas[10000:10500]
```

Line 13

We extract the content of the book in a somewhat simplistic way

```
" mes yeux. --vous avez donc vu l'empereur aussi? --il est entré chez le maréchal pendant que
j'y étais. --et vous lui avez parlé? --c'est-à-dire que c'est lui qui m'a parlé, monsieur, dit
dantès en souriant. --et que vous a-t-il dit? --il m'a fait des questions sur le bâtiment, sur
l'époque de son départ pour marseille, sur la route qu'il avait suivie et sur la cargaison qu'il
portait. je crois que s'il eût été vide, et que j'en eusse été le maître, son intention eût été
de l'acheter; mais je lu"
```

```
import spacy
from spacy import displacy

nlp = spacy.load("fr_core_news_sm")
doc = nlp(dumas[15000:17000])
# displacy.render(doc, style="ent", jupyter=True)
```

The named entity recognition provided by default in general-purpose libraries is often underwhelming; it is frequently necessary to supplement the default rules with *ad hoc* rules specific to each corpus.

In practice, named entity recognition was recently used by Etalab to pseudonymize administrative documents. This involves identifying certain sensitive information (such as civil status, address, etc.) through entity recognition and replacing it with pseudonyms.

4.2 Text Data Classification: The `Fasttext` Algorithm

`Fasttext` is a single-layer neural network developed by Meta in 2016 for text classification and language modeling. As we will see, this model serves as a bridge to more refined forms of language modeling, although `Fasttext` remains far simpler than large language models (LLMs). One of the main use cases of `Fasttext` is supervised text classification: determining a text's category. For example, identifying whether a song's lyrics belong to the rap or rock genre. This is a supervised model because it learns to recognize *features*—in this case, pieces of text—that lead to good prediction performance on both training and test sets.

The concept of a *feature* might seem odd for text data, which is inherently unstructured. For structured data, as discussed in the modeling section, the approach was straightforward: features were observed variables, and the classification algorithm identified the best combination to predict the label. With text data, we must build features from the text itself—turning unstructured data into structured form. This is where the concepts we've covered so far come into play.

`FastText` uses a *"bag of n-grams"* approach. It considers that features are derived not only from words in the corpus but also from multiple levels of n-grams. The general architecture of `FastText` looks like this:

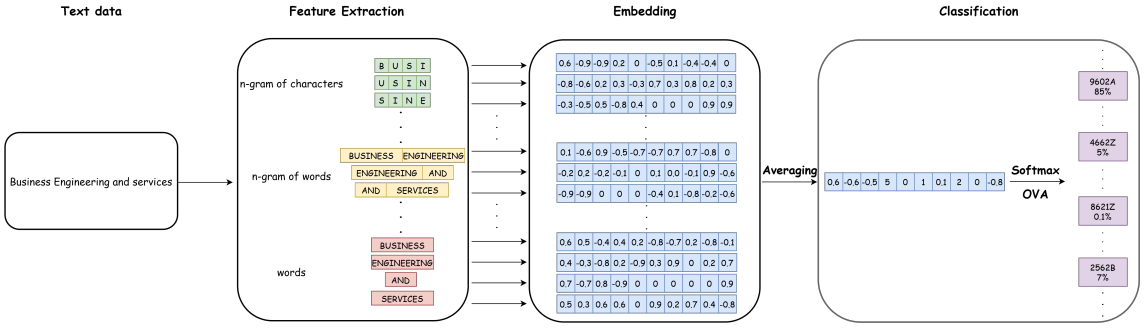


Figure 1: Diagram of `FastText` architecture

What interests us here is the left side of the diagram—*"feature extraction"*—since the *embedding* part relates to concepts we will cover in upcoming chapters. In the figure's example, the text *"Business engineering and services"* is tokenized into words as we've seen earlier. But `Fasttext` also creates multiple levels of n-grams. For instance, it generates word bigrams: *"Business engineering"*, *"engineering and"*, *"and services"*; and also character four-grams like *"busi"*, *"usin"*, and *"sine"*. Then, `Fasttext` transforms all these items into numeric vectors. Unlike the term frequency representations we've seen, these vectors are not based on corpus frequency (as in document-term matrices) but are word embeddings. We'll explore this concept in future chapters.

`Fasttext` is widely used in public statistics, as many textual data sources need to be classified into aggregated nomenclatures.

```
Erreur lors de l'appel API : 503 Server Error: Service Unavailable for url: https://codification-ape-test.lab.sspcloud.fr/predict?nb_echos_max=3&prob_min=0&text_feature=data%20scientist
```

Informations additionnelles

i Python environment

This site was built automatically through a `Github` action using the `Quarto` reproducible publishing software (version 1.10.18).

The environment used to obtain the results is reproducible via uv. The `pyproject.toml` file used to build this environment is available on the `linogaliana/python-datascientist` repository

```
[project]
name = "python-datascientist"
version = "0.1.0"
description = "Source code for Lino Galiana's Python for data science course"
readme = "README.md"
requires-python = "⩾3.13,<3.14"
dependencies = [
```

```
    "altair>=6.0.0",
    "cartiflette>=0.2",
    "contextily==1.6.2",
    "duckdb>=0.10.1",
    "folium>=0.19.6",
    "gdal==3.11.4",
    "graphviz==0.20.3",
    "great-tables>=0.12.0",
    "gt-extras>=0.0.8",
    "ipykernel>=6.29.5",
    "jupyter>=1.1.1",
    "jupyter-cache>=1.0.0",
    "kaleido>=0.2.1",
    "langchain-community>=0.3.27",
    "loguru==0.7.3",
    "markdown>=3.8",
    "nbclient>=0.10.0",
    "nbformat>=5.10.4",
    "nltk>=3.9.1",
    "openpyxl>=3.1.5",
    "pandas>=3.0",
    "pip>=25.1.1",
    "plotly>=6.1.2",
    "plotnine>=0.15",
    "polars>=1.8.2",
    "py7zr>=1.1.3",
    "pyarrow>=17.0.0",
    "pynsee>=0.1.8",
    "python-dotenv>=1.0.1",
    "python-frontmatter>=1.1.0",
    "pywaffle>=1.1.1",
    "requests>=2.32.3",
    "scikit-image>=0.24.0",
    "scikit-learn>=1.8.0",
    "scipy>=1.13.0",
    "seaborn>=0.13.2",
    "selenium<4.39.0",
    "spacy>=3.8.4",
    "webdriver-manager>=4.0.2",
    "wordcloud==1.9.3",
]

[tool.uv.sources]
gdal = [
  { index = "gdal-wheels", marker = "sys_platform == 'linux'" },
  { index = "geospatial_wheels", marker = "sys_platform == 'win32'" },
]

[[tool.uv.index]]
name = "geospatial_wheels"
url = "https://nathanjmcdougall.github.io/geospatial-wheels-index/"
explicit = true

[[tool.uv.index]]
name = "gdal-wheels"
url = "https://gitlab.com/api/v4/projects/61637378/packages/pypi/simple"
explicit = true
```

```
[dependency-groups]
dev = [
    "nb-clean>=4.0.1",
]
```

To use exactly the same environment (version of `Python` and *packages*), please refer to the documentation for `uv`.

Bibliography